

Layer 04: Marts (Gold)

Script: `sql/04_marts.sql` | **Status:** Complete and verified | **Next:** Python notebook (EDA, modeling)

1. Purpose

Turn the clean core layer into analysis-ready tables: the exposure base, point-in-time model features, monthly PMPM, and the claim-lag triangle. This is the layer the Python notebook queries directly.

2. What this layer produces

Table / view	Grain
<code>fact_member_month</code>	One row per member per month they are considered covered (real rows plus filled single-month gaps)
<code>mart_claim_features</code>	One row per claim (mirrors <code>fact_claim</code>), with leakage-safe engineered features added
<code>mart_monthly_member_months</code>	One row per month -- the PMPM denominator
<code>mart_monthly_claims</code>	One row per month per claim type -- paid dollars and count
<code>mart_pmpm</code> (view)	Joins the two above; PMPM computed live, never stored as a stale number
<code>mart_lag_triangle</code>	One row per (service month, lag in months) -- the completion-factor/IBNR input

3. D4, resolved: the coverage-gap rule

Profiling found 51% of members had at least one missing month but never measured whether those gaps were isolated or long runs. This layer measured it directly before deciding:

	Count
Isolated single-month gaps (present month immediately before and after)	37,283
Gap-months that are part of a longer run	2,750
Total gap-months	40,033

Decision: an isolated single-month gap is treated as a load miss and filled (the member is considered continuously covered through it, flagged `is_filled_gap = 1`). A gap that is part of a run of two or more consecutive missing months is left absent -- treated as a real coverage lapse, not an error to paper over.

`fact_member_month` reconciliation:

	Count
Actually reported member-months	1,202,281
Filled gap member-months	37,283
Total member-months in the exposure table	1,239,564

92.9% of all gap-months were isolated and filled; the remaining 6.9% were real, multi-month lapses and were correctly left out of exposure.

4. `mart_claim_features`: point-in-time safety

"Point-in-time safe" means every engineered feature for a claim uses only that same member's **earlier** claims, never a later one -- the property a real forecast depends on, since at prediction time the future genuinely isn't available.

- `prior_claims_12mo`: built with a self-join, not a correlated subquery, counting each member's claims strictly before this claim's `service_date` and within the preceding 12 months.
- `days_since_last_claim`: built with `LAG() OVER (PARTITION BY member_sk ORDER BY service_date)`, the gap since that member's previous claim.

Both were checked against `fact_claim`'s row count (985,000) and matched exactly.

5. PMPM and lag triangle: the design confirmed on real numbers

January 2022, by claim type:

Claim type	Claims	Total paid	PMPM
Inpatient	2,260	\$47,306,650	\$1,776.71
Emergency	2,520	\$10,153,899	\$381.35
Outpatient	5,980	\$7,431,174	\$279.09
Pharmacy	4,158	\$754,375	\$28.33
Preventive	2,441	\$736,217	\$27.65

Inpatient PMPM is **63x** Pharmacy's, from a fraction of the claim volume -- the cost concentration designed into the data, now expressed as a real, queryable business metric rather than a raw claim count.

Monthly totals show the designed trend and seasonality: claims and dollars climb through 2022, with November jumping to \$70.3M from October's \$51.2M -- the winter uplift.

6. Issues hit while building this layer, and how each was closed

#	Symptom	Root cause	Fix	Verification
1	<code>mart_claim_features</code> had engineered features that were technically present but meaningless for one group of rows	Every orphan claim (<code>unmatched_member = 1</code>) shares one placeholder <code>member_sk</code> (the <code>UNKNOWN</code> row in <code>dim_member</code>). The <code>prior_claims_12mo</code> self-join groups by <code>member_sk</code> , so for orphans it was counting other, unrelated fake patients' claims as if they belonged to one shared history. Real members were never affected, since their <code>member_sk</code> values never collide with each other or with <code>UNKNOWN</code>	Added an explicit <code>unmatched_member</code> flag to <code>mart_claim_features</code> , and set <code>prior_claims_12mo</code> and <code>days_since_last_claim</code> to <code>NULL</code> for every flagged row, rather than leaving a number that looked valid but wasn't	<code>SELECT SUM(unmatched_member), SUM(unmatched_member=1 AND prior_claims_12mo IS NOT NULL), SUM(unmatched_member=1 AND days_since_last_claim IS NOT NULL)</code> returned 24,830 / 0 / 0 -- every orphan flagged, none still carrying a contaminated value
2	An earlier run of this same logic produced a <code>mart_claim_features</code> table with 1 row instead of 985,000, despite the INSERT reporting success	Not conclusively identified -- most likely a second, still-open database session interfering. Superseded: the final run described in this document was executed as a single, isolated script end to end and reconciled correctly at every checkpoint	Rebuilt in one uninterrupted run	<code>rows_in_mart_claim_features = distinct_claim_sk_in_mart = rows_in_fact_claim = 985,000</code> , with explicit <code>row_count_matches = 1</code> and <code>claim_key_count_matches = 1</code> boolean checks built into the script itself
3	Five warnings on <code>CREATE TABLE</code> for the chronic flag columns	<code>TINYINT(1)</code> is deprecated-but-still-working MySQL syntax (the display width has been	Changed to plain <code>TINYINT</code>	Warnings no longer appear

#	Symptom	Root cause	Fix	Verification
		ignored since MySQL 8.0.19)		
4	Rerunning the script's one-time index (<code>idx_member_service</code>) threw error 1061 on every subsequent run	A plain <code>ALTER TABLE ... ADD INDEX</code> has no "if not exists" guard in standard MySQL syntax	Replaced with a conditional block: check <code>information_schema.statistics</code> for the index first, and only run the <code>ALTER TABLE</code> via a prepared statement if it's missing	The script now reruns cleanly from a single file with no manual steps to skip

7. Final reconciliation, as run

```

rows_in_mart_claim_features | distinct_claim_sk_in_mart | rows_in_fact_claim |
row_count_matches | claim_key_count_matches
985000                | 985000                | 985000                | 1
| 1

flagged_rows | orphan_rows_with_prior_claims | orphan_rows_with_last_claim_gap
24830        | 0                               | 0

```

Both checks are now built into the script itself (Section 6), so any future rerun re-verifies correctness automatically rather than relying on someone remembering to check.

8. The ER diagram

The final diagram (`docs/er_diagram.png`) was generated directly from the live database using MySQL Workbench's reverse-engineering feature, not hand-drawn -- it reflects exactly what exists in MySQL and cannot drift out of sync with the schema. `mart_pmpm` is shown with a distinct icon and color, marking it as a view rather than a stored table. The three standalone aggregate tables (`mart_monthly_claims`, `mart_monthly_member_months`, `mart_lag_triangle`) correctly show no relationship lines, since they hold no foreign keys.

9. What this layer deliberately does not do

- No frequency modeling, no severity modeling, no backtest -- that is the Python layer
- No completion factors calculated yet -- `mart_lag_triangle` is the *input* to that calculation, not the calculation itself
- Large claims are not yet capped or pooled for modeling purposes -- a modeling decision for Python, not a data-cleaning one

10. Interview one-liners

- **30 sec:** "The marts layer builds the exposure table and the model-ready features. I measured gap run-lengths before deciding a fill rule, rather than guessing, and built every feature to only look backward in time, which is what makes a forecast honest."

- **On the orphan-feature bug:** "A shared placeholder key for unmatched records meant a self-join was silently blending unrelated people's histories. I flagged those rows explicitly and nulled the contaminated values, then wrote the reconciliation check into the script itself so it's verified on every run, not just the first one."
 - **On engineering discipline:** "I built in a conditional index check so the pipeline reruns cleanly end to end with no manual steps to remember -- a script someone else could run without reading my build history first."
-

SQL and database design: complete

Four layers, each documented and committed: staging, profiling, core, marts. The schema is final and every cleaning and feature-engineering decision is evidence-backed and verified in the script itself.

Next: the Python notebook

The pivot point of the project. First step: connect via `pandas.read_sql()`, pull `mart_claim_features` and `mart_pmpm`, and confirm the row counts and PMPM numbers match what MySQL already shows -- before any EDA begins.